

Règles d'associations

1. Les variables doivent être catégorielles
2. C'est du « comptage »
3. Relations entre modalités de variables étudiées
4. Résultats « faciles d'interprétations »
5. Générations des règles moins faciles à gérer...

Règles d'associations

Age	Yeux	cheveux		Age	Yeux	cheveux
25	bleus	noirs		≤ 30	bleus	noirs
20	bleus	blonds		≤ 30	bleus	blonds
45	bruns	roux		> 30	bruns	roux
18	verts	noirs		≤ 30	verts	noirs
48	bruns	roux		> 30	bruns	roux
32	verts	roux		> 30	verts	roux
39	bleus	roux		> 30	bleus	roux
21	bleus	blonds		≤ 30	bleus	blonds
28	bruns	blonds		≤ 30	bruns	blonds
45	bruns	roux		> 30	bruns	roux

$Age \leq 30 \rightarrow Yeux_bleus$ 3/10 (30% du dataset)

$Age > 30 \rightarrow Yeux_bleus$ 1/10 (10% du dataset)

Au sein du groupe $Age \leq 30$, 3/4 (75% du groupe) ont les yeux bleus

Au sein du dataset 40% ont les yeux bleus

$\rightarrow$ Probabilité d'avoir les yeux bleus plus importante au sein du groupe ciblé vs dataset

Règles d'associations

Age	Yeux	cheveux		Age	Yeux	cheveux
25	bleus	noirs		≤ 30	bleus	noirs
20	bleus	blonds		≤ 30	bleus	blonds
45	bruns	roux		> 30	bruns	roux
18	verts	noirs		≤ 30	verts	noirs
48	bruns	roux		> 30	bruns	roux
32	verts	roux		> 30	verts	roux
39	bleus	roux		> 30	bleus	roux
21	bleus	blonds		≤ 30	bleus	blonds
28	bruns	blonds		≤ 30	bruns	blonds
45	bruns	roux		> 30	bruns	roux

$\text{Age} \leq 30 \rightarrow \text{Yeux_bleus} \quad (3/10)$

Support = $3/10$ (30%)

Confidence = $3/4$ (75%)

Lift = $(3/4)/(4/10) = 1,875$

Règles d'associations

Avantages

- Données manquantes ne sont pas un problème
- Interprétabilité des résultats
- Permet de visualiser des biais « cachés »

Désavantages

- Variables corrélées : beaucoup de règles « polluantes »
→ Règles redondantes générées
- Filtres nécessaires pour sélectionner les règles d'intérêt
- Temps de calculs et mémoire

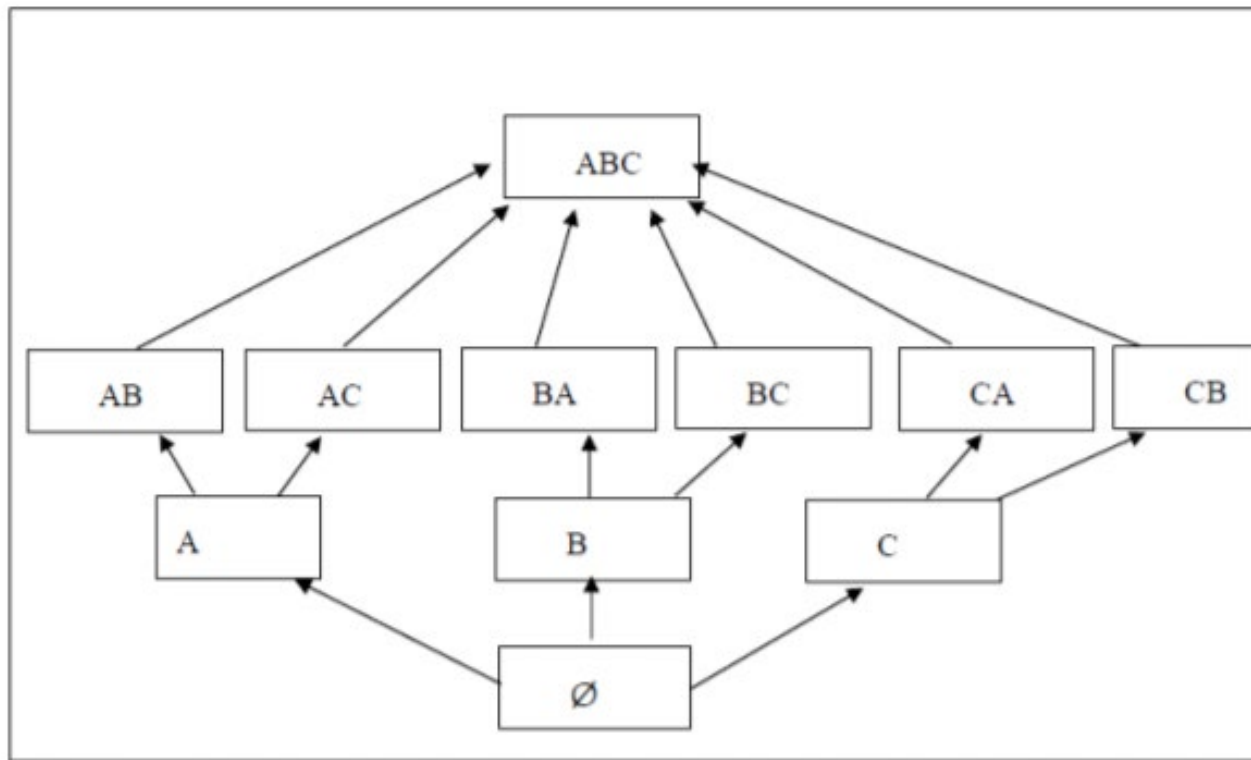
Règles d'associations

1. Ne considérer que les variables catégorielles
2. Etude des corrélations si on veut catégoriser (binarisations recommandées).
3. Dataset de variables catégorielles prêt pour étude : il faut le transformer afin d'obtenir des colonnes de modalités en 0/1
4. La génération des règles d'associations peut commencer

Algorithme Apriori

Scanner la base pour tester le support des candidats

Point faible: génération des itemsets...Beaucoup!



2 étapes:

- Itemsets Fréquents
- Règles

- Si $\{A\}$ n'est pas fréquent alors $\{AB\}$ ne peut pas l'être
- si $\{A,B\}$ est fréquent alors $\{A\}$ et $\{B\}$ le sont

Règles d'associations

Librairies python utilisées

- ***Mlxtend*** (algorithme *Apriori* pour FrequentItemsets et règles)
- Pandas
- Numpy
- matplotlib

Applications

Génération des règles d'associations

A (antécédent) $\rightarrow$ B (conséquent)

A et B {items1, item2,....}

Métriques:

Confidence n_{AB}/n_A

Support n_{AB}/n

Lift $P_{AB}/P_B.P_A$

Applications

1. Avoir un Dataset de type « transaction »

Exemple:

	Apple	Corn	Dill	Eggs	Ice cream	Kidney Beans	Milk	Nutmeg	Onion	Unicorn	Yogurt
0	False	False	False	True	False	True	True	True	True	False	True
1	False	False	True	True	False	True	False	True	True	False	True
2	True	False	False	True	False	True	True	False	False	False	False
3	False	True	False	False	False	True	True	False	False	True	True
4	False	True	False	True	True	True	False	False	True	False	False

- Utilisation de *Crosstab* de Pandas

`Pandas.crosstab()`

- Utilisation de *TransactionEncoder*

`from mlxtend.preprocessing import TransactionEncoder`

- One Hot Encoding

`from sklearn.preprocessing import OneHotEncoder`

Applications

Crosstab de Pandas

Source dataframe

make	body_style	drive_wheels	num_doors	curb_weight
mazda	hatchback	fwd	two	2385
volvo	sedan	rwd	four	2912
toyota	sedan	fwd	four	2140
mitsubishi	hatchback	fwd	two	1944
volkswagen	sedan	fwd	two	2261
mazda	sedan	fwd	four	2410
mazda	hatchback	rwd	two	2380
peugot	sedan	rwd	four	3197
mazda	sedan	fwd	four	1945
nissan	sedan	fwd	two	1918

Basic Usage

```
pd.crosstab(df.make, df.body_style)
```

body_style	convertible	hardtop	hatchback	sedan	wagon
make					
honda	0	0	7	5	1
mazda	0	0	10	7	0
mitsubishi	0	0	9	4	0
nissan	0	1	5	9	3
subaru	0	0	3	5	4
toyota	1	3	14	10	4
volkswagen	1	0	1	9	1
volvo	0	0	0	8	3

TransactionEncoder

« Liste épicerie »

	Apple	Bananas	Beer	Chicken	Milk	Rice
0	True	False	True	True	False	True
1	True	False	True	False	False	True
2	True	False	True	False	False	False
3	True	True	False	False	False	False
4	False	False	True	True	True	True
5	False	False	True	False	True	True
6	False	False	True	False	True	False
7	True	True	False	False	False	False

Applications

One Hot Encoding

Création de nouvelles variables binaires

Get_dummies

Dataset de variables catégorielles:

```
df_processed = pd.get_dummies(df, prefix_sep="__")
```

Utiliser: *drop_first=True* pour éviter de dupliquer les variables binaires

Applications

2. Algorithme Apriori: génération des *FrequentItemsets*

apriori(df, min_support=0.6, use_col_names=true, max_len=None, Verbose=0, mow_memory=false)

Exemple:

	support	itemsets
0	0.8	(Eggs)
1	1.0	(Kidney Beans)
2	0.6	(Milk)
3	0.6	(Onion)
4	0.6	(Yogurt)
5	0.8	(Eggs, Kidney Beans)
6	0.6	(Onion, Eggs)
7	0.6	(Kidney Beans, Milk)
8	0.6	(Onion, Kidney Beans)
9	0.6	(Kidney Beans, Yogurt)
10	0.6	(Onion, Kidney Beans, Eggs)

a. Filtrage des résultats

*frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(**lambda** x: len(x))*

	support	itemsets	length
0	0.8	(Eggs)	1
1	1.0	(Kidney Beans)	1
2	0.6	(Milk)	1
3	0.6	(Onion)	1
4	0.6	(Yogurt)	1
5	0.8	(Eggs, Kidney Beans)	2
6	0.6	(Onion, Eggs)	2
7	0.6	(Kidney Beans, Milk)	2
8	0.6	(Onion, Kidney Beans)	2
9	0.6	(Kidney Beans, Yogurt)	2
10	0.6	(Onion, Kidney Beans, Eggs)	3

Applications

a. Filtrage des résultats

frequent_itemsets[(frequent_itemsets['length'] == 2) & (frequent_itemsets['support'] >= 0.8)]

	support	itemsets	length
5	0.8	(Eggs, Kidney Beans)	2

frequent_itemsets[frequent_itemsets['itemsets'] == {'Onion', 'Eggs'}]

	support	itemsets	length
6	0.6	(Onion, Eggs)	2